

課外主題：LLM 推論最佳化

115-1 深度學習於海事應用 | 彈性進階主題，自主學習用

Contents

這堂課要回答的問題	2
一、學理與手算	2
步驟 1：推論其實有兩個階段	2
手算 ①：記憶體頻寬決定的速度上限	2
步驟 2：KV cache——省下重算，換來吃記憶體	2
手算 ②：KV cache 有多大	2
步驟 3：批次策略——別讓 GPU 空轉	3
手算 ③：靜態批次浪費多少	3
步驟 4：改注意力的形狀——MHA / MQA / GQA	3
手算 ④：三種注意力的 KV cache 比較	3
步驟 5：模型壓縮——量化、稀疏、蒸餾	4
手算 ⑤：int8 對稱量化	4
步驟 6：推測解碼——用小模型先猜，大模型一次驗	4
手算 ⑥：期望能省多少	4
步驟 7：放不下就拆——平行化	5
手算 ⑦：管線氣泡	5
二、實作	5
本課的技術現實要先講清楚	5
實作 A：KV cache 有沒有，差多少 (NumPy 手寫)	5
實作 B：int8 量化與誤差 (NumPy 手寫)	6
實作 C：推測解碼的接受 / 拒絕 (NumPy 手寫)	7
實作 D：框架版對照 (PyTorch，僅示範認得介面)	7
三、AI 時代，人在這裡做什麼	8
1. 定義「可以拿什麼換什麼」	8
2. 每一項最佳化都要用同一組輸入驗過	8
3. 判斷瓶頸在哪，不要優化錯地方	8
四、自我檢測 (自己算，答案在下面)	9
五、來源	9

這堂課要回答的問題

模型訓練好了、答案也對了，為什麼還是不能上線？

因為上線看的不是「會不會答」，是「多快答完、一次能服務幾個人、一張卡放不放得下」。這三件事跟模型準不準無關，跟**記憶體**有關。

本主題七個步驟，每一步都有一個可以拿紙筆算的數字。**算過一次，比記十個名詞有用。**

怎麼用這份教材：先看學理與手算，每個手算例都自己算一次；再把三支 NumPy 程式貼到 Colab 跑，確認跑出來的數字和你手算的對得上；最後看第三節「人在這裡做什麼」。全部做完大約 2–3 小時。

一、學理與手算

步驟 1：推論其實有兩個階段

階段	在做什麼	卡在哪	決定什麼指標
Prefill (預填)	把整段 prompt 一次平行算完	算力 (compute-bound)	TTFT，第一個字多久出現
Decode (解碼)	一次產生一個 token，逐字接龍	記憶體頻寬 (memory-bandwidth-bound)	TPS，每秒幾個字

關鍵認知：**產生階段的瓶頸是記憶體頻寬，不是算力**。每產生一個 token，GPU 就得把**整份模型權重**從顯示記憶體搬到運算單元一次。

手算 ①：記憶體頻寬決定的速度上限

7B 模型、fp16 (每個參數 2 bytes)、GPU 記憶體頻寬約 2 TB/s。問：decode 階段理論上每秒最多幾個 token？

權重大小 = $7 \times 10^9 \times 2 \text{ bytes} = 14 \text{ GB}$

每個 token 要搬一次 $\rightarrow 14 \text{ GB} \div 2 \text{ TB/s} = 7 \text{ 毫秒}$

上限 = $1 \div 0.007 \approx 143 \text{ tokens/秒}$

這個數字跟 GPU 有幾 TFLOPS 完全無關。換一張算力兩倍但頻寬相同的卡，decode 不會變快。

步驟 2：KV cache——省下重算，換來吃記憶體

逐字接龍時，每產生一個新 token 都要對「前面所有 token」做注意力。如果每次重算，第 n 個 token 要算 n 次；把每一層算過的 Key 與 Value 存起來 (**KV cache**)，就只要算新的那一個。

代價是這份 cache 會**隨序列長度與批次大小線性膨脹**。

手算 ②：KV cache 有多大

Llama-2-7B：32 層、32 個 KV head、head_dim 128、fp16。問：每個 token 的 KV cache 多大？4096 tokens 呢？batch 8 呢？

每 token = 2 (K 與 V) × 32 層 × 32 heads × 128 × 2 bytes
= 524,288 bytes = 512 KB

4096 tokens → 2.15 GB / 每個請求
batch 8 × 4096 → 17.2 GB

一張 80 GB 的卡，扣掉 14 GB 權重，剩下的幾乎都被 KV cache 吃掉——這就是為什麼 batch 開不大。

PagedAttention：把 cache 切成固定大小的塊，像作業系統的虛擬記憶體分頁一樣按需配置，解決「為最壞情況預留長度」造成的碎片浪費。**Prefix caching**：多個請求共用同一段開頭（例如同一份 system prompt）時，那段 cache 只算一次。

步驟 3：批次策略——別讓 GPU 空轉

策略	做法	問題
靜態批次	湊滿一批一起跑，全部跑完才換下一批	輸出長度不一，短的做完只能空等
動態批次	設時間窗與批次上限，時間到就送出	改善排隊，沒解決長度不齊
連續批次 (in-flight)	誰做完誰離開，空位立刻補新請求	目前主流，vLLM 與 TensorRT-LLM 都用

手算 ③：靜態批次浪費多少

一批 4 個請求，輸出長度分別是 10、100、20、50 個 token。問：靜態批次的 GPU 利用率是多少？

實際要算 = 10 + 100 + 20 + 50 = 180 格
實際配置 = 最長的 100 × 4 = 400 格
利用率 = 180 / 400 = 45% → 閒置 55%

一半以上的算力在等那個最長的請求。連續批次把這 55% 拿回來。

步驟 4：改注意力的形狀——MHA / MQA / GQA

既然 KV cache 是瓶頸，就從源頭把它變小：讓多個 query head 共用同一組 key/value head。

手算 ④：三種注意力的 KV cache 比較

條件同手算 ②，只改 KV head 數。

做法	KV head 數	每 token	相對 MHA
MHA 多頭注意力	32	524,288 bytes	1
GQA 分組查詢 (8 組)	8	131,072 bytes	1/4
MQA 多查詢 (共用 1 組)	1	16,384 bytes	1/32

做法	KV head 數	每 token	相對 MHA
----	-----------	---------	--------

MQA 省最多但品質掉最多，GQA 是折衷——這就是為什麼近年的模型幾乎都用 GQA。

FlashAttention 走另一條路：不改模型結構，把注意力的多個步驟融合成一個 kernel，用晶片上的 SRAM 分塊計算，減少與顯示記憶體之間的來回搬運。不需要重新訓練。

步驟 5：模型壓縮——量化、稀疏、蒸餾

量化：把權重從 FP16 降到 INT8 或 INT4。

7B 模型：FP16 約 14 GB → INT4 約 3.5 GB (四分之一)

記憶體變四分之一，搬運量也變四分之一——由步驟 1 可知，**decode 速度直接受益**。常見方法：**GPTQ**、**AWQ** (兩者都是訓練後量化，不必重訓)。

手算 ⑤：int8 對稱量化

權重 [0.80, -1.20, 0.35, 0.05]，量化到 int8 (範圍 -127 ~ 127)。問：scale 是多少？還原後誤差多大？

$$\text{scale} = \max|w| / 127 = 1.20 / 127 = 0.009449$$

$$w = +0.80 \rightarrow q = \text{round}(0.80/0.009449) = +85 \rightarrow \text{還原 } +0.8031 \text{ (誤差 } +0.0031 \text{)}$$

$$w = -1.20 \rightarrow q = -127 \rightarrow \text{還原 } -1.2000 \text{ (誤差 } 0 \text{)}$$

$$w = +0.35 \rightarrow q = +37 \rightarrow \text{還原 } +0.3496 \text{ (誤差 } -0.0004 \text{)}$$

$$w = +0.05 \rightarrow q = +5 \rightarrow \text{還原 } +0.0472 \text{ (誤差 } -0.0028 \text{)}$$

最大絕對值那一個永遠沒有誤差，小的那些誤差比例最大。這正是 AWQ 那類方法要處理的問題——重要的權重給更高精度。

稀疏：剪掉接近零的權重。NVIDIA Ampere 以後支援 **2:4 結構化稀疏** (每四個裡固定兩個為零)，最多 2 倍加速。**知識蒸餾**：用大模型的輸出去訓練小模型，讓小模型學到大模型的行為。

步驟 6：推測解碼——用小模型先猜，大模型一次驗

1. 小的 **draft 模型**一口氣猜 k 個 token
2. 大的**驗證模型**把這 k 個平行驗一次
3. 從第一個猜錯的地方截斷，接受前面猜對的部分

輸出與大模型單獨生成完全一致——這是數學上保證的，不是近似。

手算 ⑥：期望能省多少

draft 一次提 k 個 token，每個被接受的機率 α 。期望每次驗證產出的 token 數 = $(1 - \alpha^{k+1}) / (1 - \alpha)$

k	α	期望產出
4	0.8	3.36 個
4	0.5	1.94 個
8	0.8	4.33 個

k 從 4 加到 8，期望只從 3.36 升到 4.33——猜越多邊際效益遞減，而 draft 模型的成本是線性增加的。所以 k 通常取 4 上下。 α 從 0.8 掉到 0.5，期望幾乎砍半——draft 模型跟主模型像不像，比猜幾個更重要。

步驟 7：放不下就拆——平行化

做法	怎麼拆	代價
張量平行 管線平行	同一層切開放到多張卡 不同層放不同卡，像產線	每一層都要跨卡通訊 產生「氣泡」，要用 microbatch 填
Prefill / Decode 分離	兩階段送到不同硬體池，各自最佳化	架構複雜，但兩階段的瓶頸本來就不同

手算 ⑦：管線氣泡

氣泡比例 = (段數 $\square$ 1) / (microbatch 數 + 段數 $\square$ 1)

4 段、8 個 microbatch $\rightarrow 3/11 = 27\%$

4 段、32 個 microbatch $\rightarrow 3/35 = 9\%$

microbatch 切得越細，氣泡越小。代價是每個 microbatch 太小又吃不滿 GPU——又是一個取捨。

二、實作

本課的技術現實要先講清楚

本課程用 TensorFlow 2，但 LLM 推論的生態系 (vLLM、TensorRT-LLM、GPTQ、AWQ) 幾乎都在 PyTorch 上。所以這個主題的實作分兩層：

- 手寫版用 NumPy——不依賴任何框架，看得到每一個數字怎麼來
- 框架版用 PyTorch / transformers——認出生態系的介面，知道去哪裡找

手寫版才是這堂課的重點，框架版是讓學生知道業界長什麼樣。

實作 A：KV cache 有沒有，差多少 (NumPy 手寫)

```
import numpy as np
```

```
D, L = 64, 200    # head_dim, 生成長度
```

```
def decode_no_cache(x0):
    """ 沒有 cache : 每一步都把前面所有 token 的 K、V 重算一次。 """
    toks, flops = [x0], 0
    for _ in range(L):
        X = np.stack(toks)          # (n, D)
        K = X @ np.eye(D); V = X @ np.eye(D)
        flops += 2 * X.shape[0] * D * D  # 重算 K 與 V 的成本
        q = toks[-1]
        a = np.exp(K @ q); a /= a.sum()
        toks.append(a @ V)
    return flops
```

```
def decode_with_cache(x0):
    """ 有 cache : 只算新的那一個 token 的 K、V。 """
    toks, K, V, flops = [x0], [], [], 0
    for _ in range(L):
        x = toks[-1]
        K.append(x); V.append(x)
        flops += 2 * D * D          # 只算一個
        Km = np.stack(K)
        a = np.exp(Km @ x); a /= a.sum()
        toks.append(a @ np.stack(V))
    return flops
```

```
x0 = np.random.randn(D)
a, b = decode_no_cache(x0), decode_with_cache(x0)
print('沒有 cache %.2e · 有 cache %.2e · 省了 %.1f 倍' % (a, b, a / b))
```

預期結果：省下的倍數約等於 $(L+1)/2$ ，因為重算的成本是 $1+2+\dots+L$ ，有 cache 是 L 。要學生自己改 L 再跑一次，確認倍數跟著變——這比背「KV cache 可以加速」有用。

實作 B：int8 量化與誤差 (NumPy 手寫)

```
import numpy as np

def quantize_int8(w):
    scale = np.abs(w).max() / 127
    q = np.round(w / scale).astype(np.int8)
    return q, scale

def dequantize(q, scale):
    return q.astype(np.float32) * scale
```

```
w = np.array([0.80, -1.20, 0.35, 0.05], dtype=np.float32)
q, s = quantize_int8(w)
w2 = dequantize(q, s)
print('scale', s)
print('量化後', q)
print('還原 ', w2)
print('誤差 ', w2 - w)
print('記憶體 %d bytes → %d bytes' % (w.nbytes, q.nbytes))
```

要學生做的：把 w 換成一個含有極端值的向量（例如把 -1.20 改成 -50 ），再看其他三個數字的誤差變化。這一步就能自己推出「離群值會毀掉整組量化」，也就是 AWQ 那類方法要解決的問題。

實作 C：推測解碼的接受 / 拒絕（NumPy 手寫）

```
import numpy as np

rng = np.random.default_rng(學號末兩碼) # 個人化，與本課其他作業一致

def speculative_round(k, alpha):
    """draft 提 k 個，逐一驗證，遇到第一個不接受就截斷。"""
    accepted = 0
    for _ in range(k):
        if rng.random() < alpha:
            accepted += 1
        else:
            break
    return accepted + 1 # 加上驗證模型自己補的那一個

for k, a in ((4, 0.8), (4, 0.5), (8, 0.8)):
    trials = [speculative_round(k, a) for _ in range(100000)]
    print('k=%d α=%.1f 模擬 %.2f 理論 %.2f'
          % (k, a, np.mean(trials), (1 - a ** (k + 1)) / (1 - a)))
```

模擬值要對得上手算 ⑥ 的理論值——對不上就是公式或程式其中之一理解錯了，這正是驗證的意義。

實作 D：框架版對照（PyTorch，僅示範認得介面）

```
# use_cache 開關對速度的影響
out = model.generate(**inputs, max_new_tokens=128, use_cache=True) # 預設
out = model.generate(**inputs, max_new_tokens=128, use_cache=False) # 關掉重算

# 量化：載入時直接指定
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
cfg = BitsAndBytesConfig(load_in_4bit=True)
model = AutoModelForCausalLM.from_pretrained(name, quantization_config=cfg)

# 連續批次：vLLM
```

```
from vllm import LLM, SamplingParams
llm = LLM(model=name) # 內建 PagedAttention 與 continuous batching
outs = llm.generate(prompts, SamplingParams(max_tokens=128))
```

不要求學生跑得起來（需要 GPU 與較大的下載量），要求的是看得懂這三段各自對應前面哪一個步驟。

三、AI 時代，人在這裡做什麼

本課程先前定的角色是「實驗的設計者與稽核者」。放到推論最佳化上，具體是三件事：

1. 定義「可以拿什麼換什麼」

每一項技巧都是取捨，而取捨的標準只有人能定：

技巧	換到什麼	付出什麼
量化 INT4	記憶體 1/4、速度變快	準確率下降（多少要量）
MQA	KV cache 1/32	生成品質下降
連續批次	吞吐量大幅提升	單一請求延遲可能變差
推測解碼	延遲下降	輸出完全不變，但要多養一個 draft 模型

「能不能掉 1% 準確率換 3 倍速度」——這個問題沒有技術答案，只有業務答案。

2. 每一項最佳化都要用同一組輸入驗過

這是最容易被跳過、也最容易出事的一步。最佳化看起來都有效，因為速度一定會變快；變慢的是品質，而品質不會自己報錯。

規定的驗收表，四欄缺一不可：

版本	TTFT	TPS	峰值記憶體	品質指標
原始 fp16				
+ KV cache				
+ INT4 量化				
+ 連續批次				

只報速度不報品質的最佳化結果，一律不接受。推測解碼是唯一可以宣稱「品質完全不變」的，因為那是數學保證——其餘都要拿數字說話。

3. 判斷瓶頸在哪，不要優化錯地方

手算 ① 已經證明：decode 階段換算力更強的卡沒有用。人的價值在於先算出瓶頸，再決定要不要花錢。AI 可以幫你套用每一種技巧，但「這台機器現在卡在哪」要你自己量。

四、自我檢測 (自己算，答案在下面)

1. 某 13B 模型 fp16，GPU 頻寬 1.5 TB/s，decode 的 token/s 上限是多少？
2. 某模型 40 層、8 個 KV head、head_dim 128、fp16，2048 tokens 的 KV cache 多大？
3. 一批 5 個請求輸出長度 20、200、40、60、80，靜態批次利用率多少？
4. 權重 [2.5, 0.4, 0.9, 3.0] 量化到 int8，scale 與各自的還原誤差？
5. draft 提 k=6、 $\alpha=0.7$ ，期望每次驗證產出幾個 token？
6. 8 段管線、16 個 microbatch，氣泡佔多少？

解答 1. $13 \times 2 = 26 \text{ GB}$ ； $26/1500 = 17.3 \text{ ms} \rightarrow$ 約 58 tokens/s 2. $2 \times 40 \times 8 \times 128 \times 2 = 163,840 \text{ bytes/token} \times 2048 = 336 \text{ MB}$ 3. $(20+200+40+60+80) / (200 \times 5) = 400/1000 = 40\%$ 4. $\text{scale} = 3.0/127 = 0.023622$ ；q = 106、17、38、127；還原 2.5039、0.4016、0.8976、3.0000 5. $(1 - 0.7^6)/(1 - 0.7) = (1 - 0.0824)/0.3 = 3.06$ 個 6. $7/23 = 30\%$

五、來源

- The Roadmap to Mastering LLM Inference Optimization, Machine Learning Mastery <https://machinelearningmastery.com/the-roadmap-to-mastering-llm-inference-optimization/>
- 文中另引 Modular 的 LLM Inference Handbook 與 NVIDIA 的推論最佳化深入說明

本份為**彈性進階主題**，不列入每週進度、不計分、不佔課堂時間，提供給想再往前一步的同學自主學習。七個手算例都可以拿紙筆算，三支手寫程式在 Colab 貼上就能跑，不需要 GPU。手算例的數字皆以程式重算驗證過；GPU 頻寬 2 TB/s 為 A100 等級的概略值，換卡要重算。